

Python Course for Physicists (And everyone else!)

Chunk 4: Lists and Tuples

By Jonathan Bollig

08.10.2025

Live Coding: Structuring More Data in a Single Variable: Lists

- How we can do it now:

```
measurement_1 = 3.13  
measurement_2 = 5.83  
measurement_3 = 7.22
```

- A better way to do it:

```
measurements = [3.13, 5.83, 7.22]
```

→ A „list“

- Can contain anything:

```
cool_list = [3.5, 3j, "hi", True, None]
```

- Even a list in a list:

```
cool_list2 = [[1, 3, 5], "hi", True]
```

Live Coding: Slicing / Indexing

- Access single values in a list: `[index]`
→ 0 – Indexed!
- Counting from back: `[-index]`
→ 1 – Indexed!
- Accessing multiple values:
`[start_index : end_index]`
→ Start is inclusive
→ End is exclusive
- From beginning / end:
`[start_index:]` / `[: end_index]`

```
my_list = [100, 50, 0, 99, 4, 42]
print(my_list)
print(my_list[0])
```

```
[100, 50, 0, 99, 4, 42]
100
```

```
my_list = [100, 50, 0, 99, 4, 42]
print(my_list)
print(my_list[-1])
```

```
[100, 50, 0, 99, 4, 42]
42
```

```
my_list = [100, 50, 0, 99, 4, 42]
print(my_list)
print(my_list[1:3])
```

```
[100, 50, 0, 99, 4, 42]
[50, 0]
```

```
my_list = [100, 50, 0, 99, 4, 42]
print(my_list)
print(my_list[:3])
```

```
[100, 50, 0, 99, 4, 42]
[100, 50, 0]
```

Live Coding: Changing Lists:

- Sliced / Indexed parts can be altered:

```
my_list = [100, 50, 0, 99, 4, 42]
print(my_list)
my_list[0] = 01e6
print(my_list)
```

```
[100, 50, 0, 99, 4, 42]
[1000000.0, 50, 0, 99, 4, 42]
```

```
my_list = [100, 50, 0, 99, 4, 42]
print(my_list)
my_list[0:3] = [1e1, 1e2, 1e3]
print(my_list)
```

```
[100, 50, 0, 99, 4, 42]
[10.0, 100.0, 1000.0, 99, 4, 42]
```

Live Coding: Arithmetic Operators on Lists:

- “+”: concatenate →

```
list1 = [1, 2, 3, 4]
list2 = ["a", "b", "c"]

added_list = list1+list2
print(added_list)
```

[1, 2, 3, 4, 'a', 'b', 'c']

- “*”: repeat (with integer) →

```
list1 = ["a", "b", "c"]
list2 = 3*list1
print(list2)
```

['a', 'b', 'c', 'a', 'b', 'c', 'a', 'b', 'c']

- Get length of list:

```
my_list = [100, 50, 0, 99]
print(len(my_list))
```

4

Live Coding: General Functions on Lists

- Mostly unimportant for us, but good to know.
- Only important:
→ `append()` :

```
my_list = [100, 50, 0, 99, 4, 42]
print(my_list)
my_list.append(10)
print(my_list)

[100, 50, 0, 99, 4, 42]
[100, 50, 0, 99, 4, 42, 10]
```

Method	Description
<code>append()</code>	Adds an element at the end of the list
<code>clear()</code>	Removes all the elements from the list
<code>copy()</code>	Returns a copy of the list
<code>count()</code>	Returns the number of elements with the specified value
<code>extend()</code>	Add the elements of a list (or any iterable), to the end of the current list
<code>index()</code>	Returns the index of the first element with the specified value
<code>insert()</code>	Adds an element at the specified position
<code>pop()</code>	Removes the element at the specified position
<code>remove()</code>	Removes the first item with the specified value
<code>reverse()</code>	Reverses the order of the list
<code>sort()</code>	Sorts the list

[https://www.w3schools.com/python/python_ref_list.asp]

Live Coding: Functions on Numeric List:

- If list only contains numbers:

→max():

```
li = [1, 43, 2,3]  
max(li)
```

43

→min()

```
li = [1, 43, 2,3]  
min(li)
```

1

→sum()

```
li = [1, 43, 2,3]  
sum(li)
```

49

Tuples: Similar to Lists, but less flexible

- Created with “()” instead of “[]”
- Accessed similarly:

```
my_set = (10, 14, 19, "hi there")
print(my_set[0])
print(my_set[1:3])
print(my_set[2:])
```

```
10
(14, 19)
(19, 'hi there')
```

- However: Cannot be changed:

```
my_set = (10, 14, 19, "hi there")
my_set[0] = 1
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[85], line 2
      1 my_set = (10, 14, 19, "hi there")
----> 2 my_set[0] = 1

TypeError: 'tuple' object does not support item assignment
```

- For the nerds: Lists are mutable type, sets are immutable.

Dictionaries

- Often used in python
- Not so important for scientific purposes (at least at the beginning)
- Short intro: Like lists, but with named indices:

```
person_info = {"name": "Jonathan",  
               "age": 26,  
               "height [m]": 1.8}  
print(person_info["name"])
```

Jonathan

- Another type of data: „sets“, haven't used them once.

Good Luck with your Tasks 😊

Other languages:

proper description of the error

Python:



[<https://www.reddit.com/r/ProgrammerHumor/comments/j2kanm/python/>]